Forms input invalidation: Many websites use forms that are filled with information by website users and submitted to the server. The server then validates the input and saves it to the database. The validation process is sometimes delegated to the client browser, or to the database server. If these validations are either not strong enough, or not properly programmed, they can leave security holes that can be exploited by hackers. For example, if a field such as the PAN number is mandatory, and if the validation for duplicate entries is not done properly, the hacker can programmatically submit forms with dummy PAN numbers, thus flooding the database with bogus entries. This can eventually help the hacker in planting a denial of service (DoS) attack, by simply querying the page asking for entries that do not exist.

Code exploitation: While this is a bit similar to the previous vulnerability, there is some difference in the way hackers exploit it. Often, programmers make assumptions while setting limits for various user inputs. Typical examples are that the user name should never exceed 50 characters, or that numeric values will always be positive, etc. These assumptions are dangerous from the security standpoint, because hackers can exploit them. For example, by filling the name field with 100 characters, and thus putting a stress on the datasets, or by providing negative integers in the numeric fields to create incorrect calculation results.

All the attacks mentioned above are used by novice hackers, and following good programming practices can help stop them. Let's now take a look at technically advanced attacks, which are also common today.

Cookie poisoning: As explained earlier, cookies are small information snippets residing in the browser (on the client machine's hard drive), and are used to store user session-specific information. It's the cookie that remembers our shopping cart contents, our preferences and the previous log-in information, in order to provide a rich Web experience. While it is not very easy to tamper with a cookie, an advanced hacker can gain control of it and manipulate its content. Poisoning is achieved via a Trojan or a virus, which sits in the background and keeps forging cookies to gather a user's personal information and send it to the hacker. Besides this, the virus can also alter the contents of cookies to cause serious problems, such as submitting shopping cart contents in such a way as to deliver the purchased items to a dummy address accessible to the hacker, or to let the browser connect to advertisement servers, which helps the hacker gain money, etc. If session information is stored in the cookie, advanced hackers can gain access to it and steal the session, causing a man-in-the-middle attack.

Session hijacking: A Web server talks to multiple browsers at the same time, to take requests in and to deliver the requested content. While each connection is made, the Web server needs to have a way to maintain the uniqueness for each connection. It uses session tokens for this—dynamically generated text strings, which are factors of an IP address, the date, time, etc. Hackers can steal this token either by guessing programmatically or sniffing on the network, or by performing a client-side script attack on a victim computer. Once stolen, this token can be used to create a fake Web request and steal the victim user's session and information.

URL query-string tampering: Websites that pull data from a database server and show it on the Web page are often found to use query-strings in the main URL. For example, if the website URL is *http://www.a.com/*, it may use *http://www.a.com/showdata?field1=10&field2=15* to pass *field1* and *field2* as parameters, with their respective values, to the database—and the resultant output is provided to the browser in the form of a Web page. Having this query-string format exposed so easily, it is possible for users to edit and alter field values beyond the expected limits, or fill them with junk characters. It can further result in users gaining access to information they are not supposed to get. In the worst case, if the field values are *userid* and *password*, a brute-force dictionary attack can be used to gain system-level access, merely over HTTP.

Cross-site scripting: This is the most common vulnerability in Web technology, attracting XSS (cross-site scripting) attacks on all major and famous websites. It has been found that a large number of websites are vulnerable to this attack, even today. This vulnerability is a result of improper programming practices and the unavailability of appropriate security measures in a Web infrastructure. As we know, a client browser maintains its own security in terms of not allowing website contents and the website cookies to be accessed by anyone, except by the users themselves. In this case, the vulnerabilities in a Web application let hackers inject client-side code into the page accessed by users. This code is typically written using JavaScript. To understand this, consider a page that takes the user name as input, and writes back on the screen "Welcome username". Let us then suppose the input box is filled up with JavaScript instead, such as what follows:

```
"<script> alert ('You are in trouble') </script>
```

Here, the Web page may end up executing the script tags, showing the dialogue box message "You are in trouble". This can be further exploited by the hacker, by simply poisoning the cookie, stealing the session and injecting this code into the victim user's browser. Upon doing so, the JavaScript code would run in the victim's